

CyberAudit & Solutions

Rapport final — Résultats du projet & Enseignements tirés

Holberton School Dijon | Tommy JOUHANS & James ROUSSEL | Juillet 2026

Nom du projet	CyberAudit & Solutions : plateforme SaaS d'audit cybersécurité B2B
Chef de projet	Tommy JOUHANS (Front-End) / James ROUSSEL (Back-End)
Date	6 juillet 2026
Période	Stage 5 : du 6 au 17 juillet 2026 (5 semaines)
Statut	MVP déployé : Railway (backend) + Vercel (frontend)

1. Achèvement du projet

1.1 Résumé des résultats

CyberAudit & Solutions a atteint ses objectifs MVP dans la fenêtre de développement de 5 semaines prévue. L'application est entièrement déployée et opérationnelle en production.

Objectif initial	Résultat	Statut
MVP full-stack : Django + React	Backend Django REST Framework avec 5 apps (accounts, audits, reports, training, notifications). Frontend React 19 + Vite. Les deux déployés et accessibles.	Atteint
Authentification JWT + RBAC	SimpleJWT avec tokens access/refresh, blacklist de token à la déconnexion, contrôle d'accès basé sur les rôles (client / admin) strict sur chaque endpoint.	Atteint
Génération PDF asynchrone	Pipeline Celery + Redis + WeasyPrint. L'admin déclenche la génération, le serveur répond 202 Accepted, le worker génère le PDF de façon asynchrone et notifie le client.	Atteint
Couverture de tests ≥ 80 %	131 tests backend (100 % de couverture sur les 5 apps). 331 tests frontend répartis sur 19 fichiers Vitest. CI/CD GitHub Actions imposant les deux à chaque push.	Dépassé
Déploiement en production	Backend en ligne sur Railway avec PostgreSQL + Redis. Frontend en ligne sur Vercel avec déploiement automatique sur push vers main. HTTPS forcé sur les deux.	Atteint
Renforcement de la sécurité	DOMPurify + middleware CSP + RBAC + clés primaires UUID + anti-énumération (404 au lieu de 403) + recalcul du score côté serveur + throttling.	Atteint
Tests E2E Cypress	Prévu pour le Sprint 7. Hors périmètre de la livraison MVP. Les tests unitaires et d'intégration couvrent les chemins critiques.	Reporté

1.2 Indicateurs de performance clés

Métrique	Résultat
Couverture de tests backend	100 % (131 tests, 1 242 lignes couvertes)
Couverture de tests frontend	331 tests répartis sur 19 fichiers Vitest
Vélocité moyenne des sprints	95 % (38 / 40 tâches planifiées terminées)
Bugs ouverts à la livraison	0 (6 bugs résolus pendant les sprints)
Endpoints API	25+ endpoints REST avec RBAC complet
Pipeline CI/CD	Lint + Tests + Build sur chaque PR
Délai du premier déploiement	Backend sur Railway depuis mai 2026
Temps de génération PDF	2-5 secondes (asynchrone, non bloquant)

1.3 Éléments en suspens

Ces éléments ont été identifiés mais restent hors périmètre du MVP. Ils sont prévus pour une version future.

Élément	Action requise	Responsable
Suite de tests E2E Cypress	Implémenter des tests end-to-end couvrant les parcours utilisateur complets (inscription, soumission d'audit, téléchargement PDF)	Tommy
Endpoint d'export de données (RGPD Art.20)	Ajouter GET /api/auth/me/export renvoyant les données utilisateur en JSON	James
Suppression automatique des comptes inactifs	Tâche Celery Beat : anonymiser les comptes inactifs depuis 3+ ans	James
Préchargement HSTS	Définir SECURE_HSTS_SECONDS=31536000 dans les variables d'environnement Railway	James
aria-label sur les boutons icône seule	Ajouter des labels accessibles aux boutons icône autonomes pour les lecteurs d'écran	Tommy
Table de journal d'audit de connexion	Table SQL dédiée pour les événements de connexion (détection d'intrusion)	James

2. Enseignements tirés

La construction de CyberAudit nous a beaucoup appris en 4 semaines. Cette section documente ce qui a bien fonctionné, les défis rencontrés et comment ils ont été résolus, ainsi que les principes clés que nous emporterons pour la suite.

2.1 Ce qui a bien fonctionné

Fait	Impact	Pour les prochains projets
Défense en profondeur : auto-échappement React + DOMPurify + Content Security Policy actifs simultanément.	Zéro vulnérabilité XSS signalée. Le CSP a bloqué l'injection de script externe en production (confirmé par la suite de tests).	Appliquer une sécurité en couches dès le premier jour. Chaque couche est indépendante ; si l'une échoue, les autres protègent toujours.
Recalcul du score de sécurité côté serveur : <code>compute_score_and_grade()</code> dans <code>reports/services.py</code> ignore tout score soumis par le client.	Le client ne peut pas manipuler sa note d'audit. Intégrité du score de sécurité garantie quel que soit ce que le frontend envoie.	Ne jamais faire confiance à une entrée client pour quoi que ce soit de critique pour la sécurité. Valider et recalculer côté serveur.
Asynchrone par défaut : Celery + Redis gère toutes les tâches de plus d'une seconde.	L'API répond toujours en moins de 200 ms. Les utilisateurs reçoivent un 202 Accepted instantanément et sont notifiés quand le PDF est prêt.	Concevoir l'asynchrone dès le départ pour toute tâche pouvant dépasser 1 seconde. Le retrofit est douloureux.
Séparation claire front/back avec des contrats d'API convenus avant le codage. Tommy possédait tout le frontend, James tout le backend.	Zéro surprise d'intégration. Un mock localStorage a permis au frontend d'avancer sans attendre la disponibilité du backend.	Définir les interfaces JSON et les signatures d'endpoints avant le début du codage. Élimine le désalignement entre équipes.

2.2 Défis et résolutions

Problème	Cause racine	Résolution	Prévention
Railway effaçait les PDF générés à chaque déploiement.	Système de fichiers éphémère de Railway : le conteneur est reconstruit à zéro à chaque déploiement, effaçant tous les fichiers écrits à l'exécution.	Attachement d'un volume Railway persistant monté sur <code>MEDIA_ROOT</code> . Les PDF survivent désormais aux déploiements.	Tout fichier écrit à l'exécution (uploads, assets générés) doit utiliser un volume persistant ou un stockage externe (S3) en environnement conteneurisé.
localStorage provoquait des conflits de session entre plusieurs onglets.	localStorage est partagé entre tous les onglets de la même origine. Une déconnexion dans l'onglet A ne se propageait pas à l'onglet B.	Migration vers sessionStorage. Chaque onglet a sa propre session ; fermer un onglet la nettoie automatiquement.	Utiliser sessionStorage pour les tokens d'authentification sauf exigence explicite d'état persistant entre onglets. Bénéficie sécurité additionnel : nettoyage à la fermeture.

Problème	Cause racine	Résolution	Prévention
Notre propre polling de statut PDF déclenchait le rate limiter côté serveur.	ReportViewerPage interrogeait HEAD /api/reports/pdf/ toutes les quelques secondes. Avec plusieurs onglets ouverts, le seuil de 5 req/min du scope login était dépassé.	Remplacement par un polling intelligent de 30 secondes, s'arrêtant automatiquement une fois pdfReady=true.	Toujours tenir compte de son propre polling dans les calculs de rate limit. Le backoff exponentiel ou les server-sent events sont plus robustes à l'échelle.

2.3 Principes clés pour les projets futurs

#	Principe	Application dans CyberAudit	Comment l'appliquer la prochaine fois
1	Défense en profondeur	Auto-échappement React + DOMPurify + CSP : trois couches XSS indépendantes.	Empiler plusieurs contrôles de sécurité indépendants. Ne jamais dépendre d'un seul mécanisme.
2	Ne jamais faire confiance au client	Score de sécurité calculé côté serveur. Signature JWT vérifiée à chaque requête. RBAC appliqué dans get_queryset().	Traiter toute entrée client comme non fiable. Valider, assainir et recalculer côté serveur toutes les valeurs sensibles.
3	Asynchrone par défaut	Celery gère la génération de PDF et les e-mails. L'API répond toujours en moins de 200 ms.	Toute opération de plus d'une seconde doit passer par une file de tâches. Concevoir le chemin asynchrone avant le repli synchrone.
4	Convenir des contrats avant de coder	Interfaces JSON et signatures d'endpoints convenues au Stage 3. Le frontend utilisait des mocks contre le même contrat.	Rédiger le contrat d'API (formats requête/réponse, codes de statut) en document partagé avant la première ligne de code.
5	Tester tôt, tester partout	100 % de couverture backend + 331 tests frontend. La CI bloque les merges en cas d'échec de tests.	Fixer un seuil de couverture dès le sprint 1. Rattraper les tests après coup est trois fois plus difficile que de les écrire en parallèle du code.

2.4 Conclusion

Quatre semaines. Vingt améliorations. Un vrai MVP de production. CyberAudit a été construit dans les mêmes contraintes qu'un projet professionnel : deux développeurs, un calendrier fixe, un déploiement réel, et de vraies exigences de sécurité. Les défis rencontrés et résolus — systèmes de fichiers éphémères, gestion de session, rate limiting auto-infligé — sont les mêmes que ceux rencontrés dans l'industrie. Les principes que nous en avons tirés ne sont pas théoriques : ils sont ancrés dans des échecs de production et leurs correctifs. Nous les emportons avec nous.

3. Rétrospective d'équipe

Cette rétrospective a été menée à la fin du Stage 4 (3 juillet 2026) par les deux membres du projet : Tommy JOUHANS (Frontend) et James ROUSSEL (Backend). Son objectif est d'évaluer honnêtement la performance de l'équipe, d'identifier ce qui a conduit au succès, et de définir des améliorations concrètes pour le prochain projet.

Date	3 juillet 2026 (fin du Stage 4)
Participants	Tommy JOUHANS / James ROUSSEL
Format	Rétrospective de fin de sprint : Start / Stop / Continue + 3 questions guides
Animateur	Tommy JOUHANS
Preneur de notes	James ROUSSEL

3.1 Qu'est-ce qui a bien fonctionné en équipe ?

Pratique	Observation de l'équipe
Répartition claire des responsabilités	Tommy possédait 100 % du frontend ; James 100 % du backend. Aucune ambiguïté sur qui était responsable d'un fichier ou d'une décision donnée. Cela a évité le travail dupliqué et les reports de responsabilité en cas de bug.
Contrat d'API en premier	Les signatures d'endpoints, formats de requête/réponse et codes de statut HTTP ont été convenus par écrit avant que l'un ou l'autre n'écrive du code d'implémentation. Le frontend a été construit contre des mocks du même contrat, faisant de l'intégration une formalité plutôt qu'une négociation.
Qualité imposée automatiquement par la CI/CD	GitHub Actions exécutait lint, tests et build à chaque pull request. Aucun développeur ne pouvait merger une branche cassée. Cela a éliminé entièrement le problème du « ça marche sur ma machine » et gardé la branche main toujours déployable.
Couverture de tests dès le sprint 1	Les deux membres se sont engagés à écrire les tests en même temps que le code, pas après. Résultat : 100 % de couverture backend et 331 tests frontend livrés sans coup de collier final. La qualité était une habitude, pas une tâche de dernière minute.
Architecture asynchrone conçue tôt	La décision d'utiliser Celery pour toutes les opérations de plus d'une seconde a été prise en phase de conception. Le chemin asynchrone n'a donc jamais été rétrofité — il était le défaut. Génération de PDF, e-mails et notifications en ont tous bénéficié.
Communication ouverte sur les blocages	Les deux développeurs signalaient les blocages immédiatement (crash Railway, rate limit atteint, gel de déploiement) plutôt que de réessayer silencieusement. Cela a permis de diagnostiquer les causes racines plus vite et évité de perdre des jours à courir après des symptômes.

3.2 Quels défis avons-nous rencontrés, et comment ont-ils été résolus ?

Ces défis concernent la dimension humaine et processus de chaque problème ; les causes techniques racines sont détaillées en section 2.2.

Défi	Résolution	Enseignement processus
Surprises d'infrastructure en production (FS éphémère Railway, crash de facturation, blocage GitHub Pages)	Chaque problème a été diagnostiqué à partir des logs de déploiement, pas par supposition. L'équipe a adopté un principe : lire les logs d'abord, chercher l'erreur exacte ensuite, ne jamais supposer. Chaque correctif a été documenté immédiatement dans le README du projet.	L'infrastructure de production devrait être validée au Sprint 1, pas au Sprint 5. Un smoke test de 30 minutes sur la plateforme réelle aurait détecté le système de fichiers éphémère et les problèmes de facturation trois semaines plus tôt.
Décision de stockage de session (localStorage vs sessionStorage)	Le bug multi-onglets a été reproduit localement, la cause racine identifiée (état localStorage partagé), et le correctif (sessionStorage) proposé, revu et mergé en un seul sprint. La décision a été prise conjointement, pas unilatéralement.	Les choix d'implémentation critiques pour la sécurité (stockage de token, flux d'authentification) devraient être revus par les deux membres de l'équipe avant la première ligne de code, pas découverts via un bug.
Rate limiting auto-infligé par le polling	Une fois les erreurs 429 tracées jusqu'à notre propre frontend, l'équipe s'est accordée sur le correctif (polling de 30 secondes avec arrêt automatique) lors d'un point de synchronisation rapide. Le correctif a été mergé le jour même.	Les seuils de rate limit devraient être écrits dans un document partagé dès le début du projet. Les développeurs frontend et backend doivent tous deux connaître les limites que leur code doit respecter.
Swagger /api/docs/ renvoyant 404 (drf_spectacular manquant dans INSTALLED_APPS)	Le développeur backend a identifié l'incohérence entre <code>urls.py</code> et <code>settings.py</code> , ajouté la ligne manquante, et le correctif était en ligne en quelques minutes.	Toute bibliothèque importée dans le code doit apparaître dans <code>INSTALLED_APPS</code> ou l'équivalent du manifeste de configuration. Une checklist des paires « le code l'utilise, la config le déclare » préviendrait cette classe d'erreur.
Conflit de merge lors de l'intégration (Stages/ Stage4/README.md)	Les deux versions (diagramme mermaid depuis HEAD, image PNG depuis remote) ont été examinées. La décision de garder les deux a été prise immédiatement, aucun contenu n'a été perdu. Le conflit a été résolu en moins de 10 minutes.	Les branches de fonctionnalité longue durée augmentent le risque de conflit de merge. Des branches plus courtes (max 2 jours) et des merges plus fréquents vers main réduiraient cette friction.

3.3 Comment pouvons-nous améliorer la collaboration sur les prochains projets ?

Les actions d'amélioration suivantes ont été convenues par les deux membres de l'équipe.

Action d'amélioration	Pourquoi	Responsable	Quand
Déployer en production dès le Sprint 1	Les surprises d'infrastructure (FS éphémère, identifiants, intégrations CI) sont bien moins coûteuses à corriger au début qu'en milieu de projet.	Les deux	Sprint 1
Cross-review d'au moins une PR par sprint sur le domaine de l'autre	Chaque développeur ne revoit actuellement que son propre domaine. Les cross-reviews diffusent la connaissance et détectent les bugs transversaux (ex : polling frontend contre les rate limits backend).	Les deux	Chaque sprint
Rédiger un document partagé de rate-limits et contrats de sécurité au lancement	Le bug de rate-limit sur le polling est survenu car le frontend ne connaissait pas les seuils de throttle du backend. Un document partagé l'aurait évité.	James	Sprint 1
Plafonner les branches de fonctionnalité à 2 jours avant merge vers main	Les branches longues causent des conflits de merge et reportent les surprises d'intégration. Des branches plus courtes gardent les deux développeurs synchronisés.	Les deux	Sprint 2
Ajouter les tests E2E Cypress dès le Sprint 3 (non reportés)	Les tests unitaires confirment que les composants individuels fonctionnent. Seuls les tests E2E confirment que le parcours utilisateur complet fonctionne de bout en bout en production.	Tommy	Sprint 3
Maintenir un .env.example vivant documentant chaque variable requise	La rotation de DATABASE_URL sur Railway a causé un crash en production en partie parce que le mapping des variables n'était pas clairement documenté pour les deux membres.	James	Sprint 1
Réserver 10 % de chaque sprint à la dette technique et à la documentation	HSTS, l'endpoint d'export RGPD et les aria-labels ont été reportés car chaque sprint était 100 % dédié aux fonctionnalités. Un tampon dédié éviterait l'accumulation.	Les deux	Chaque sprint

3.4 Retours documentés

Les retours individuels suivants ont été enregistrés textuellement à la clôture de la réunion de rétrospective.

Tommy JOUHANS

Travailler avec un contrat clair et un pipeline CI fiable a rendu le travail frontend solide dès le premier jour. Je n'ai jamais eu à me demander si mon code allait casser le backend — les tests et les interfaces convenues s'en chargeaient. Ce que je ferais différemment : faire tourner Cypress dès la semaine deux, sans le reporter. L'absence de tests E2E était la seule partie du projet qui semblait inachevée à la livraison.

James ROUSSEL

Les décisions d'architecture backend prises au Stage 3 — Celery, RBAC, blacklist JWT — ont parfaitement tenu sous pression. Rien n'a dû être repensé en cours de sprint. Les problèmes d'infrastructure Railway étaient frustrants mais formateurs : les environnements conteneurisés se comportent différemment des machines de développement, et il faut le savoir avant de déployer. Pour les prochains projets, j'insisterais pour un smoke test complet de production à la fin du sprint un, pas du sprint cinq.